

Gen-Health Analytics -- Test Data & Testing Guide

Version: 1.0.0

Engine: main.py -> engine/risk_scorer.py

Date: 2026-06-22

Prerequisites

```
# Install Python dependencies
pip install -r requirements.txt
# or
pipenv install

# Verify the engine runs
python main.py schemas/input_schema.json
```

How to Run a Test Case

From a file

```
python main.py test_data/<filename>.json
```

From stdin

```
cat test_data/<filename>.json | python main.py
```

Capture output to file

```
python main.py test_data/<filename>.json > output.json
```

Exit codes

Code	Meaning
0	Success -- JSON printed to stdout
1	Validation error or invalid JSON -- message printed to stderr

Test Cases Overview

ID	File	Condition(s)	Purpose
----	------	--------------	---------

TC01	tc01_high_risk_diabetes.json	Diabetes	High DPF, 4 generations, strongly accelerating trend
TC02	tc02_early_onset_cvd.json	Heart Disease	Early-onset CVD (men <55), RR >= 3.0
TC03	tc03_brca_ashkenazi.json	Breast / Ovarian Cancer	BRCA1/2 via Ashkenazi ethnicity + NCCN flags
TC04	tc04_lynch_colorectal.json	Colorectal Cancer	Lynch syndrome (3+ relatives, 2+ generations)
TC05	tc05_multi_condition.json	Diabetes + CVD + Breast Cancer	Multiple simultaneous high-priority alerts
TC06	tc06_low_risk_baseline.json	Diabetes + Heart Disease	No family history -- baseline RR only
TC07	tc07_invalid_missing_field.json	--	Invalid input (empty conditions_of_interest)
TC08	tc08_accelerating_trend.json	Heart Disease	Strongly accelerating onset trend across 3 generations

TC01 -- High-Risk Diabetes

File: test_data/tc01_high_risk_diabetes.json

Scenario: 38-year-old woman with diabetes in father (gen -1, onset 50), paternal grandmother (gen -2, onset 58), maternal grandfather (gen -2, onset 64), and paternal great-grandmother (gen -3, onset 70). Lifestyle flags: HighBP, PhysInactivity, Obesity in father.

Run:

```
python main.py test_data/tc01_high_risk_diabetes.json
```

Expected Output:

Field	Expected Value
conditions.diabetes.relative_risk_ratio	~2.23
conditions.diabetes.onset_trend.trend_classification	strongly_accelerating
conditions.diabetes.onset_trend.slope	~ -10.0 (years earlier per generation)
conditions.diabetes.affected_relatives_count	4
red_flag_alerts[0].priority	high
red_flag_alerts[0].trigger_reason	Contains "Strongly accelerating"
summary.total_alerts	1
summary.high_priority_count	1

What this tests:

- DPF proxy calculation from multi-generational history
- Onset trend OLS regression with data from 3 distinct generation indexes
- strongly_accelerating classification (slope < -8 years/generation)
- Lifestyle attribution partial scoring (HighBP, PhysInactivity lift attribution fraction)

TC02 -- Early-Onset CVD

File: test_data/tc02_early_onset_cvd.json

Scenario: 42-year-old man. Father died of MI at 48 (gen -1), paternal grandfather died of heart disease at 52 (gen -2), paternal great-grandfather at 60 (gen -3), and paternal uncle at 51. All males; early-onset threshold for males is 55.

Run:

```
python main.py test_data/tc02_early_onset_cvd.json
```

Expected Output:

Field	Expected Value
conditions.heart_disease.relative_risk_ratio	~3.01
conditions.heart_disease.early_onset_flag	true
conditions.heart_disease.family_history_flag	true
conditions.heart_disease.onset_trend.trend_classification	accelerating
red_flag_alerts[0].priority	high
red_flag_alerts[0].trigger_reason	Contains "3.0" and "early-onset"
summary.high_priority_count	1

What this tests:

- CVD family history RR multiplier (2.316x)
- Early-onset penalty applied (onset < 55 for males)
- RR >= 3.0 high-priority alert threshold
- cause_of_death field recognized correctly

TC03 -- BRCA Risk / Ashkenazi Jewish

File: test_data/tc03_brca_ashkenazi.json

Scenario: 35-year-old Ashkenazi Jewish woman. Mother has breast cancer (onset 42), maternal aunt has breast cancer (onset 38), maternal grandmother has ovarian cancer (onset 55, deceased).

Run:

```
python main.py test_data/tc03_brca_ashkenazi.json
```

Expected Output:

Field	Expected Value
conditions.breast_cancer.cancer_flags	2 entries -- ethnicity flag + NCCN rule flag
cancer_flags[0].category	ethnicity
cancer_flags[0].ethnicity_modifier	10.0
cancer_flags[1].trigger_reason	Contains "2+ relatives with breast cancer before age 50"
conditions.ovarian_cancer.cancer_flags[0].trigger_reason	Contains "Any ovarian cancer"
red_flag_alerts length	3
summary.high_priority_count	3
recommended_action	Contains "BRCA1/2 testing" and "genetic counseling"

What this tests:

- Ethnicity-specific cancer risk modifier (10x BRCA multiplier for Ashkenazi)
- NCCN breast cancer rule: 2+ first/second-degree relatives with onset < 50
- NCCN ovarian cancer rule: any first/second-degree relative (no age threshold)
- Multiple cancer flags stacked on a single condition
- proband_ethnicity field parsed and applied

TC04 -- Lynch Syndrome / Colorectal Cancer

File: test_data/tc04_lynch_colorectal.json

Scenario: 45-year-old man. Father has colorectal cancer (onset 48, deceased), paternal grandfather (onset 58, deceased), paternal uncle (onset 46), paternal aunt (onset 55). Four relatives across two generations.

Run:

```
python main.py test_data/tc04_lynch_colorectal.json
```

Expected Output:

Field	Expected Value
red_flag_alerts length	3
Alert priorities	high, high, medium
First high alert trigger	Contains "3+ relatives...Lynch syndrome"
Second high alert trigger	Contains "Strongly accelerating"
Medium alert trigger	Contains "First-degree relative...before age 50"

What this tests:

- Lynch syndrome NCCN rule: ≥ 3 relatives, ≥ 2 generation indexes
- Alert generation for both count threshold AND early-onset first-degree rules simultaneously
- cause_of_death as string ("colorectal cancer") vs boolean (true)

Note: Sub-type condition names `colon\_cancer` and `rectal\_cancer` are matched separately from `colorectal\_cancer` by the trigger rules. Use `colorectal\_cancer` consistently across relatives to satisfy the Lynch syndrome count threshold.

TC05 -- Multi-Condition High Risk

File: test_data/tc05_multi_condition.json

Scenario: 50-year-old woman. Father: diabetes (onset 55) + heart disease (onset 62, deceased). Mother: breast cancer (onset 48) + diabetes (onset 60). Paternal grandfather: diabetes + heart disease. Maternal grandmother: breast cancer (deceased at 55). Sister: breast cancer (onset 44).

Run:

```
python main.py test_data/tc05_multi_condition.json
```

Expected Output:

Field	Expected Value
-------	----------------

conditions keys	diabetes, heart_disease, breast_cancer
summary.total_alerts	>= 4
summary.high_priority_count	>= 3
summary.highest_priority_condition	A condition with a high-priority alert
Breast cancer flags	NCCN 2+ rule + NCCN 3+ rule
Heart disease trend	strongly_accelerating
Diabetes alert priority	medium

What this tests:

- Parallel scoring of 3 conditions in a single call
- Alert builder prioritization across multiple conditions
- summary.highest_priority_condition selection logic
- Breast cancer NCCN rules for 2+ and 3+ relatives simultaneously

TC06 -- Low Risk / Baseline

File: test_data/tc06_low_risk_baseline.json

Scenario: 30-year-old man. Both parents and both grandparents have no recorded conditions. No lifestyle flags.

Run:

```
python main.py test_data/tc06_low_risk_baseline.json
```

Expected Output:

Field	Expected Value
conditions.diabetes.relative_risk_ratio	~1.0 (baseline, no affected relatives)
conditions.heart_disease.relative_risk_ratio	1.0
summary.high_priority_count	0
summary.medium_priority_count	0
onset_trend.trend_classification	insufficient_data (0 data points)

What this tests:

- Engine handles zero affected relatives without crashing
- Baseline RR of 1.0 returned when no family history exists
- Insufficient data path for onset trend
- No spurious alerts generated for clean family history

TC07 -- Invalid Input (Error Handling)

File: test_data/tc07_invalid_missing_field.json

Scenario: Input has an empty conditions_of_interest array (schema requires >=1 item).

Run:

```
python main.py test_data/tc07_invalid_missing_field.json
echo "Exit code: $?"
```

Expected Output:

```
Validation error: 'conditions_of_interest' must be a non-empty list of condition names.
Exit code: 1
```

Behavior	Expected
Stdout	Empty (no JSON)
Stderr	Validation error: 'conditions_of_interest' must be a non-empty list...

Exit code	1
-----------	---

What this tests:

- validate_and_parse() rejects empty conditions_of_interest
- Engine exits with code 1 on validation failure
- Error message goes to stderr, not stdout

TC08 -- Strongly Accelerating CVD Trend

File: test_data/tc08_accelerating_trend.json

Scenario: 35-year-old woman. Maternal great-grandmother: heart disease onset 68 (gen -3). Maternal grandmother: onset 55 (gen -2). Mother: onset 45 (gen -1). Maternal aunt: onset 42 (gen -1). Clear decreasing trend across three generations.

Run:

```
python main.py test_data/tc08_accelerating_trend.json
```

Expected Output:

Field	Expected Value
onset_trend.trend_classification	strongly_accelerating
onset_trend.slope	<= -10 (years earlier per generation)
relative_risk_ratio	~3.01 (family history RR x early-onset multiplier)
red_flag_alerts[0].priority	high
projected_proband_age	A number < 40

What this tests:

- OLS regression with data spanning 3 generation indexes
- strongly_accelerating threshold: slope <= -8 years/generation
- projected_proband_age extrapolation to generation index 0
- Early-onset flag for women (threshold < 65)

Running All Test Cases

```
# Run all and capture outputs
for f in test_data/tc0*.json; do
    echo "=== $f ==="
    python main.py "$f" 2>&1 | python -c "
import sys, json
try:
    d = json.load(sys.stdin)
    s = d['summary']
    print(f'  Alerts: {s["total_alerts"]} (H:{s["high_priority_count"]} M:{s["medium_priority_count"]} L:{s["low_priority_count"]})
    print(f'  Top condition: {s["highest_priority_condition"]}')
except Exception as e:
    print(f'  ERROR or non-JSON output: {e}')
"
done
```

Running the Playwright UI Tests

The UI tests require the Streamlit app to be running first:

```
# Terminal 1 -- start the app
pipenv run streamlit run app.py

# Terminal 2 -- run the tests
pipenv run pytest tests/ -v
```

Output Schema Quick Reference

The engine always returns JSON with these top-level keys:

```

{
  "analysis_version": "1.0.0",
  "analysis_timestamp": "<ISO 8601>",
  "proband_age": 38,
  "proband_sex": "female",
  "conditions": {
    "<condition_name>": {
      "relative_risk_ratio": 2.23,
      "genetic_predisposition_score": 26,
      "environmental_risk_score": 5,
      "correlation_score": 14.6,
      "affected_relatives_count": 4,
      "onset_trend": { "trend_classification": "strongly_accelerating", ... },
      "cancer_flags": [ ... ]
    }
  },
  "red_flag_alerts": [
    {
      "alert_id": "<uuid>",
      "condition": "<name>",
      "priority": "high | medium | low",
      "trigger_reason": "<string>",
      "recommended_action": "<string>"
    }
  ],
  "summary": {
    "highest_priority_condition": "<name>",
    "total_alerts": 3,
    "high_priority_count": 2,
    "medium_priority_count": 1,
    "low_priority_count": 0
  }
}

```

Alert Priority Thresholds

Priority	Triggers (checked in order)
High	Cancer red flags (NCCN) . RR $\geq$ 3.0 . Strongly accelerating trend
Medium	RR $\geq$ 2.0 . Accelerating trend . DPF $\geq$ 1.0
Low	DPF $\geq$ 0.7 . Early CVD onset . RR $\geq$ 1.5

Trend Classification Thresholds

Classification	Slope (years/generation)
strongly_accelerating	$\leq ?8$
accelerating	$?8 < \text{slope} < 0$
stable	$?2 \leq \text{slope} \leq 2$

decelerating	slope > 2
insufficient_data	< 2 distinct generation indexes

Supported Conditions

Condition Name	Detector Type	Lifestyle Flags Available
diabetes	DPF proxy + risk bands	HighBP, HighChol, Smoker, PhysInactivity, HvyAlcoholConsump, Stroke, HeartDiseaseorAttack
heart_disease	CVD family RR x early-onset	Smoking, Obesity, Sedentary_Lifestyle, High_BP, High_Cholesterol, Chronic_Stress, Diabetes
breast_cancer	NCCN rule-based	None
ovarian_cancer	NCCN rule-based	None
male_breast_cancer	NCCN rule-based	None
colorectal_cancer	NCCN rule-based	None
colon_cancer	NCCN rule-based	None
rectal_cancer	NCCN rule-based	None
hypertension	Baseline (RR 1.0)	None
Unknown	Baseline (RR 1.0)	None

Gen-Health Analytics -- preventive health informatics capstone project